

LAB 4 :Process Scheduling

Objective :To know about working principle of different CPU scheduling algorithm.

1. First Come First Serve (FCFS) CPU Scheduling Alogrithm

```
#include <stdio.h>

struct Process {
    int pid, bt, at, wt, tat;
};

int main() {
    int n, i, j;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct Process p[n], temp;
    printf("Enter burst time and arrival time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i+1);
        scanf("%d %d", &p[i].bt, &p[i].at);
        p[i].pid = i+1;
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < n-i-1; j++) {
            if (p[j].at > p[j+1].at) {
                temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
}
```

```
        p[j+1] = temp;
    }
}
}

int current_time = 0;
float sum_wt=0, avg_wt;
float sum_tat=0, avg_tat ;
printf("Process\t\tBurst Time\tArrival Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++) {
    p[i].wt = current_time;
    current_time += p[i].bt;
    p[i].tat = current_time-p[i].at;
    sum_wt += p[i].wt;
    sum_tat += p[i].tat;
    printf("P%d\t\t%d\t\t%d\t\t%d\t\t%d\n", p[i].pid, p[i].bt, p[i].at, p[i].wt, p[i].tat);
}

avg_wt = sum_wt/n;
avg_tat = sum_tat/n;
printf("\nAverage Waiting time (AWT) = %.2f", avg_wt);
printf("\nAverage Turnaround time (ATAT) = %.2f\n", avg_tat);

return 0;
}
```

NOTE: Give input value from classroom activity and check the output for each scheduling (is correct or not)as we solved in class.

2.Shortest Job First (SJF) CPU Scheduling Alogrithm

```
#include <stdio.h>

struct Process {
    int pid, bt, wt, tat;
};

int main() {
    int n, i, j;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct Process p[n], temp;
    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i+1);
        scanf("%d", &p[i].bt);
        p[i].pid = i+1;
    }

    for (i = 0; i < n-1; i++)
        for (j = i+1; j < n; j++) {
            if (p[i].bt > p[j].bt) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
```

```
}
```

```
int current_time = 0;
float sum_wt=0, avg_wt;
float sum_tat=0, avg_tat ;
printf("Process\t\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++) {
    p[i].wt = current_time;
    current_time += p[i].bt;
    p[i].tat = current_time;
    sum_wt += p[i].wt;
    sum_tat += p[i].tat;
    printf("P%d\t\t%d\t\t%d\t\t%d\n", p[i].pid, p[i].bt, p[i].wt, p[i].tat);
}
```

```
avg_wt = sum_wt/n;
avg_tat = sum_tat/n;
printf("\nAverage Waiting time (AWT) = %.2f", avg_wt);
printf("\nAverage Turnaround time (ATAT) = %.2f\n", avg_tat);
```

```
return 0;
```

```
}
```

3.Round Robin Scheduling Algorithm

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid, bt, at, wt, rt, tat;
```

```
};
```

```
int main() {  
    int n, i, j, quantum;  
    printf("Enter the number of processes: ");  
    scanf("%d", &n);  
    printf("Enter the quantum time: ");  
    scanf("%d", &quantum);  
  
    struct Process p[n];  
    printf("Enter burst time and arrival time:\n");  
    for (i = 0; i < n; i++) {  
        printf("P%d: ", i+1);  
        scanf("%d %d", &p[i].bt, &p[i].at);  
        p[i].pid = i+1;  
        p[i].rt = p[i].bt;  
    }  
  
    int current_time = 0;  
    int completed_processes = 0;  
    while (completed_processes < n) {  
        for (i = 0; i < n; i++) {  
            if (current_time >= p[i].at && p[i].rt > 0) {  
                if (p[i].rt > quantum) {  
                    current_time += quantum;  
                    p[i].rt -= quantum;  
                } else {  
                    current_time += p[i].rt;  
                }  
            }  
        }  
        completed_processes++;  
    }  
}
```

```

        p[i].wt = current_time - p[i].at - p[i].bt;
        p[i].tat = current_time - p[i].at;
        p[i].rt = 0;
        completed_processes++;
    }
}
}
}

```

```

float sum_waiting = 0, sum_turnaround = 0, avg_waiting = 0, avg_turnaround = 0;
printf("Process ID\tBurst Time\tArrival Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, p[i].at, p[i].wt, p[i].tat);
    sum_waiting += p[i].wt;
    sum_turnaround += p[i].tat;
}
avg_waiting = sum_waiting/n;
avg_turnaround = sum_turnaround/n;
printf("\nAverage Waiting time (AWT) = %.2f", avg_waiting);
printf("\nAverage Turnaround time (ATAT) = %.2f\n", avg_turnaround);
return 0;
}

```

4.Priority Scheduling Alogrithm

```

#include <stdio.h>

struct Process {
    int pid, bt, wt, tat, priority;

```

```
};
```

```
int main() {  
    int n, i, j;  
    printf("Enter the number of processes: ");  
    scanf("%d", &n);  
  
    struct Process p[n], temp;  
    printf("Enter burst time and priority no:\n");  
    for (i = 0; i < n; i++) {  
        printf("P%d: ", i+1);  
        scanf("%d %d", &p[i].bt, &p[i].priority);  
        p[i].pid = i+1;  
    }  
  
    for (i = 0; i < n-1; i++)  
        for (j = i+1; j < n; j++) {  
            if (p[i].priority > p[j].priority) {  
                temp = p[i];  
                p[i] = p[j];  
                p[j] = temp;  
            }  
        }  
    }
```

```
int current_time = 0;  
float sum_wt=0, avg_wt;
```

```
float sum_tat=0, avg_tat ;

printf("Process\t\tPriority\t\tBurst Time\tWaiting Time\tTurnaround Time\n");

for (i = 0; i < n; i++) {

    p[i].wt = current_time;

    current_time += p[i].bt;

    p[i].tat = current_time;

    sum_wt += p[i].wt;

    sum_tat += p[i].tat;

    printf("P%d\t\t%d\t\t%d\t\t%d\t\t%d\n", p[i].pid, p[i].priority, p[i].bt, p[i].wt, p[i].tat);

}


avg_wt = sum_wt/n;

avg_tat = sum_tat/n;

printf("\nAverage Waiting time (AWT) = %.2f", avg_wt);

printf("\nAverage Turnaround time (ATAT) = %.2f\n", avg_tat);


return 0;

}
```